Linear Search and Binary Search Study Guide

1. Introduction to Linear Search and Binary Search

Linear Search

Linear search is a simple searching algorithm that checks each element in a list sequentially until the target element is found or the entire list has been searched. It works on both sorted and unsorted arrays.

Key Characteristics:

- Searches elements one by one from the beginning
- Works on both sorted and unsorted arrays
- Simple to implement and understand
- No preprocessing required

Binary Search

Binary search is an efficient searching algorithm that works on sorted arrays by repeatedly dividing the search interval in half. It compares the target value with the middle element and eliminates half of the remaining elements at each step.

Key Characteristics:

- Only works on sorted arrays
- Divides the search space in half at each step
- Much faster than linear search for large datasets
- Requires the array to be sorted beforehand

2. Time Complexity Analysis and Space Complexity

Linear Search Complexity Analysis

Time Complexity:

- **Best Case:** O(1) - Element found at the first position
- **Average Case:** O(n) - Element found at the middle position
- **Worst Case:** O(n) - Element found at the last position or not found

Space Complexity: O(1) - Only uses a constant amount of extra space

Binary Search Complexity Analysis

Time Complexity using Recurrence Relation:

The recurrence relation for binary search is:

```
T(n) = T(n/2) + O(1)
```

Where:

- T(n) represents the time to search in an array of size n
- T(n/2) represents the time to search in half the array
- O(1) represents the constant time for comparison and calculation

Solving the recurrence:

- T(n) = T(n/2) + 1
- T(n/2) = T(n/4) + 1
- T(n/4) = T(n/8) + 1
- ...
- T(1) = 1

After k divisions: n/2^k = 1, which means k = $\log_2(n)$

Therefore: **Time Complexity = O(log n)**

Time Complexity Summary:

- **Best Case:** O(1) - Element found at the middle position
- **Average Case:** O(log n)
- **Worst Case:** O(log n) - Element not found or found after maximum divisions

Space Complexity:

- **Iterative approach:** O(1) - Constant space
- **Recursive approach:** O(log n) - Due to recursive call stack

3. Implementation of Linear Search and Binary Search

Linear Search Implementation

```java
public class LinearSearch {

    // Linear search method
    public static int linearSearch(int[] arr, int target) {
        // Iterate through each element in the array
        for (int i = 0; i < arr.length; i++) {
            // If element is found, return its index
            if (arr[i] == target) {
                return i;
            }
        }
        // If element is not found, return -1
        return -1;
    }

    // Main method to test linear search
    public static void main(String[] args) {
        int[] array = {64, 34, 25, 12, 22, 11, 90};
        int target = 22;

        int result = linearSearch(array, target);

        if (result != -1) {
            System.out.println("Element found at index: " + result);
        } else {
            System.out.println("Element not found in the array");
        }
    }
}
```

Binary Search Implementation

Iterative Approach

java

```java
public class BinarySearchIterative {

    // Binary search using iterative approach
    public static int binarySearch(int[] arr, int target) {
        int left = 0;
        int right = arr.length - 1;

        while (left <= right) {
            // Calculate middle index
            int mid = left + (right - left) / 2;

            // If element is found at mid
            if (arr[mid] == target) {
                return mid;
            }

            // If target is smaller, search left half
            if (arr[mid] > target) {
                right = mid - 1;
            }
            // If target is larger, search right half
            else {
                left = mid + 1;
            }
        }

        // Element not found
        return -1;
    }

    // Main method to test binary search
    public static void main(String[] args) {
        int[] array = {11, 12, 22, 25, 34, 64, 90}; // Sorted array
        int target = 25;

        int result = binarySearch(array, target);

        if (result != -1) {
            System.out.println("Element found at index: " + result);
        } else {
            System.out.println("Element not found in the array");
        }
```

```
    }
}
```

Recursive Approach

java

```java
public class BinarySearchRecursive {

    // Binary search using recursive approach
    public static int binarySearch(int[] arr, int left, int right, int target) {
        // Base case: element not found
        if (left > right) {
            return -1;
        }

        // Calculate middle index
        int mid = left + (right - left) / 2;

        // If element is found at mid
        if (arr[mid] == target) {
            return mid;
        }

        // If target is smaller, search left half
        if (arr[mid] > target) {
            return binarySearch(arr, left, mid - 1, target);
        }

        // If target is larger, search right half
        return binarySearch(arr, mid + 1, right, target);
    }

    // Wrapper method for easier calling
    public static int search(int[] arr, int target) {
        return binarySearch(arr, 0, arr.length - 1, target);
    }

    // Main method to test recursive binary search
    public static void main(String[] args) {
        int[] array = {11, 12, 22, 25, 34, 64, 90}; // Sorted array
        int target = 34;

        int result = search(array, target);

        if (result != -1) {
            System.out.println("Element found at index: " + result);
        } else {
            System.out.println("Element not found in the array");
        }
```

```
    }
}
```

Quiz Questions

1. What is the time complexity of linear search in the worst case? a) O(1) b) O(log n) c) O(n) d) O(n^2)

Answer: c) O(n)

2. Which of the following is a prerequisite for binary search? a) Array must be unsorted b) Array must be sorted c) Array must contain only positive numbers d) Array must have even number of elements

Answer: b) Array must be sorted

3. What is the space complexity of iterative binary search? a) O(1) b) O(log n) c) O(n) d) O(n^2)

Answer: a) O(1)

4. In binary search, if the middle element is greater than the target, what happens next? a) Search the right half b) Search the left half c) Return -1 d) Search both halves

Answer: b) Search the left half

5. What is the recurrence relation for binary search? a) T(n) = T(n-1) + O(1) b) T(n) = 2T(n/2) + O(1) c) T(n) = T(n/2) + O(1) d) T(n) = T(n) + O(log n)

Answer: c) T(n) = T(n/2) + O(1)

Practice Coding Questions

1. Modified Linear Search Write a Java method that performs linear search and returns all indices where the target element is found (not just the first occurrence). The method should return an ArrayList of integers.

2. Binary Search for First Occurrence Implement a binary search algorithm that finds the first occurrence of a target element in a sorted array that may contain duplicates. If the element is not found, return -1.

3. Binary Search in Rotated Array Write a Java method to search for a target element in a rotated sorted array. For example, in array [4,5,6,7,0,1,2], the original sorted array [0,1,2,4,5,6,7] was rotated.

4. Count Occurrences Using Binary Search Implement a method that uses binary search to count the total number of occurrences of a target element in a sorted array. Use binary search to find both the first

and last occurrence.

5. Search Insert Position Given a sorted array and a target value, return the index where the target would be inserted to maintain the sorted order. If the target exists in the array, return its index. Use binary search approach.